Finding the Border of a Robot Structure

Border-tracing algorithm on Blinky Blocks (VisibleSim)

Muhammad Omais Rana

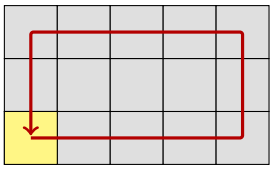
Abstract

A structure built from many small robots is hard to describe: listing every robot takes a lot of memory. Here the robots describe their structure by its *outline*. One message walks along the outline, robot after robot, and collects a short description of it. The same idea applies to the outlines of holes inside the structure. No robot sees the whole structure and there is no central computer; robots only talk to the neighbours they touch. We implemented this in VisibleSim, tested it on three structures, and added our own way of storing the outline compactly: each straight stretch of the border is packed into a single byte.

1 The problem in one picture

Each robot is a small cube that knows only its own position and can talk to the robots touching it. A robot is on the **border** if at least one of its four neighbouring cells is empty. A structure has one *outer* border and one *inner* border for each hole.

Goal: find every border and store it in as little memory as possible.



Yellow: the **starter** robot

The message walks along the border

12 border robots, described by
1 start corner + 4 one-byte segments

Figure 1: One message walks once around the border and records only the straight stretches.

2 How the robots trace a border

Every step is just a robot passing a short message to a neighbour.

1. **Choose starters.** A robot becomes a *starter* if the cell in front of it (below it) is empty and either the cell to its left is empty or the cell diagonally in front-right is occupied. This rule guarantees that every border, outer or inner, has at least one starter.

2. **Walk along the border.** A starter sends a message to its first neighbour on the border. Each robot that receives it looks for the next robot to pass it to by trying the directions in a fixed turning order, always hugging the border.

3. **Record the turns.** The message carries the list of the straight stretches seen so far. When a robot on the border sees that the walk changes direction, it closes the current stretch (direction and length) and adds it to the message as one packed byte. Straight parts cost nothing extra, only the turns are recorded.

4. **Starters compete.** Several starters may trace the same border at the same time. Each robot remembers, for every direction a message arrived from, which starter it came from. If a message arrives from a "bigger" starter (compared by x, then y) than one already seen from that direction, it is dropped. In the end only the message of the smallest starter survives.

5. **Finish.** When the message returns to its starter, the border is closed. The starter now holds the complete compact description and prints it.

Robots run independently and messages may arrive in any order. This is why several starters on one border can send messages at the same time, and why the number of messages can differ slightly from one run to another.

3 Our compact representation

The assignment asks us to propose our own way to make the border description smaller. Listing every border robot is wasteful, because most of them lie on straight stretches. Instead we store:

- the **start corner** (one position), and
- a list of **segments**, each made of a *direction* (front, right, back, left) and a *length*.

A straight stretch of any length costs a single segment, so the size of the description depends on the *number of turns* of the border and not on its length. In Figure 1 the 12 border robots are described by one start corner and 4 segments. The program prints this as "representation elements = 1 start corner + S segments" for each border.

One byte per segment. A direction needs only 2 bits and most stretches are shorter than 64 robots, so each segment is packed into a single byte:

bits 7–6	bits 5–0
direction (0 front, 1 right, 2 back, 3 left)	length -1

Because a segment is never empty, storing *length* -1 lets the 6 bits cover lengths 1 to 64. A stretch longer than 64 robots is stored as several consecutive segments of the same direction (for example 130 robots become $64 + 64 + 2$); the reader simply adds them up, so no special code is needed. The program prints the payload as "segment payload = S byte(s)".

Before this change a segment stored the direction and the length in two separate fields, which the compiler pads to 4 bytes. The packed form is therefore 4 times smaller. A corner list with two-byte coordinates needs about 2 bytes per corner, so one byte per segment makes the description about half as big as a corner list (Table 2). The saving also applies to the messages: the segment list travels inside every message and grows as the walk proceeds, so the messages become smaller too.

4 Tests

We tested three planar structures of different shapes.

Structure	Robots	Border robots	Starters	Borders found	Messages	Time	Elements
`config`	82	50	8	4	197	63.1 ms	26
`mario`	101	54	8	4	224	111.8 ms	31
`smiley`	136	70	10	6	297	77.4 ms	35

Table 1: Results measured in VisibleSim. *Border robots* have at least one empty neighbour. *Starters* start a walk. *Borders found* is the number of walks that came back to their starter. *Time* is the simulated time until the last message was handled. *Elements* is the sum of "start corner + segments" over all borders found. Message and time values come from the realtime scheduler and may vary slightly between runs.

Structure	Segments (all borders)	Unpacked (4 B each)	Corner list (2 B each)	Packed (1 B each)
`config`	22	88 B	44 B	**22 B**
`mario`	27	108 B	54 B	**27 B**
`smiley`	29	116 B	58 B	**29 B**

Table 2: Memory used by the segment lists of all borders found. The number of segments is read from the program output (`config`: 4+4+6+8, `mario`: 2+0+3+22, `smiley`: 1+1+1+2+4+20). The corner list assumes one corner per segment and two bytes per corner. The start corner of each border is the same in every variant and is not counted.

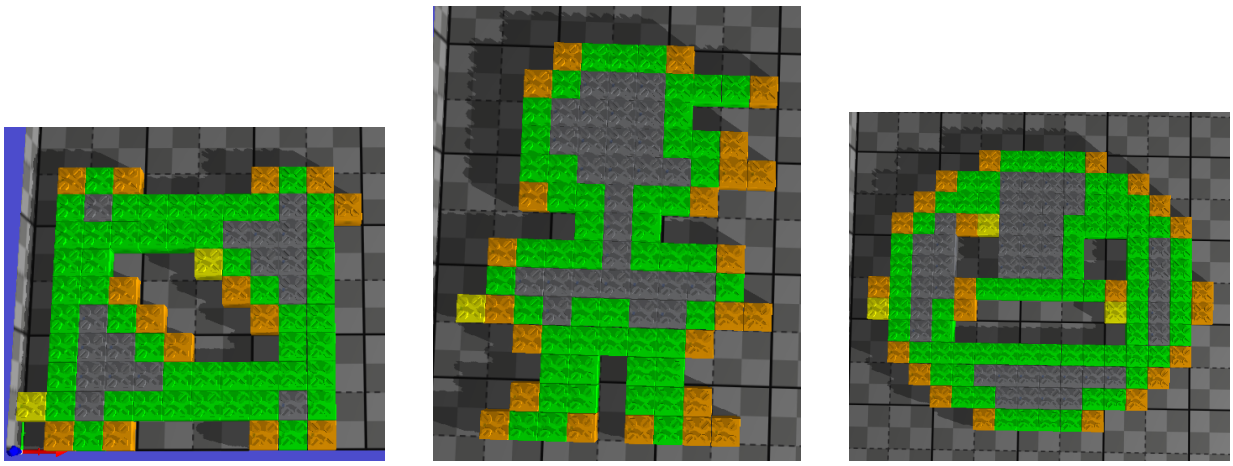
What we observe.

- *Messages.* About 4 messages per border robot (3.9, 4.1 and 4.2). This is more than one per robot because 8 to 10 starters each launch a walk, and a walk is only stopped when it meets a smaller starter. The totals stay well below the worst case $I \cdot P$ (for example 197 against $8 \times 50 = 400$).
- *Time* follows the slowest walk, not the number of robots: `mario` (101 robots) took longer than `smiley` (136 robots).
- *Size of the description.* The borders found are stored in 26, 31 and 35 elements, fewer than the 50, 54 and 70 border robots. With one byte per segment the segment lists take only 22, 27 and 29 bytes (Table 2), a quarter of the unpacked size and half of a two-byte corner list.
- *Same walk, smaller payload.* Message counts and times are the same as before the packing, because the walk itself did not change. Only the size of what each message carries became smaller.
- *Colours in the screenshots.* Yellow = starter, orange = turn, green = robot the message passed through, grey = robot no message went through (mostly the inside of the structure).

Limitations we noticed.

- Some of the borders found are very short loops: three one-segment borders in `smiley`, and in `mario` one border with no segment at all and one that only goes `LEFT x 1`, `RIGHT x 1`. They are closed by their starter, but they do not look like the outline of a whole hole.
- In the larger borders the same direction can appear twice in a row (for example `RIGHT x 2`, `RIGHT x 2`). A turn is only recorded at a robot that has an empty neighbour. At a turn where the robot has all four neighbours (an inner corner), the turn is not recorded, so the segments do not follow the walk exactly there. This is not caused by the 64-robot limit of the packed segments: the longest stretch in our tests is 7 robots.

- Because no stretch in our tests reaches 64 robots, the splitting of long stretches was never triggered in the simulations. We checked it separately with a small test program (all four directions, lengths from 1 to 65 535): the pieces always add up to the original length and each piece is between 1 and 64.
- Because of the first two points, the description can only be trusted as a compact summary of where borders pass, not as an exact outline.



`config` `mario` `smiley`

Figure 2: Borders found for each structure. Colours: starter = yellow, turn (corner) = orange, robot the message passed through = green, not visited = grey.

5 Complexity

We use these symbols: n = number of robots; P = number of robots on a border (the longest one for time); I = number of starters; S = number of straight segments of a border (the number of turns, $S \leq P$, plus extra pieces for stretches longer than 64).

	Complexity	In plain words
Communication (messages)	$O(P)$ per border when one starter wins quickly; worst case $O(I\,P)$	A message passes each border robot once. With several starters on the same border each of them may walk part of it before being stopped; in the worst case each walks all of it. Only border robots send messages, interior robots send none.
Time (parallel)	$O(P)$	The message moves one robot at a time, so a border takes as many steps as it has robots. Different borders are traced at the same time, so the longest border decides.
Memory per robot	$O(1)$	Each robot remembers at most one starter position per direction (four values). The message itself grows with the number of turns: $O(S)$, one byte per segment.
Memory of the result	$O(S)$	The starter stores a start corner and S one-byte segments (S bytes) instead of P border positions. Since $S \leq P$ and is usually much smaller, this is a real saving.

Table 3: Complexity of the border-tracing algorithm.

6 Conclusion

A single message walking along a border is enough to find and describe it, and every border, including those around holes, gets its own starter. The cost is low in messages, since only border robots take part, but the time grows with the length of the longest border because the walk is sequential. Storing the border as a start corner plus direction-and-length segments keeps the description proportional to the number of turns, and packing each segment into one byte (2 bits of direction, 6 bits of length) makes it a quarter of the unpacked size and half of a two-byte corner list. Our tests also show where the algorithm is weak: short spurious loops and turns that are not recorded at inner corners, so the result is a compact summary rather than an exact outline.